

Student Expense Tracker

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the requirements for the Student Expense Tracker, an AI-assisted web application designed to help students record their expenses, manage monthly budgets, and understand their spending habits based on the money they actually spend.

The purpose of this document is to:

- Clearly define system requirements
- Serve as a reference for developers
- Provide a foundation for system design and testing

1.2 Scope

Student Expense Tracker is a web-based system that:

- Collects and stores a student's profile, income, and expense records with category, date, and notes
- Allows students to set overall and category-wise monthly budgets
- Calculates daily, weekly, and monthly spending statistics
- Delivers budget status (percentage used, 0–100% and above) and category-wise spending charts
- Provides simple AI-generated spending insights with a short explanation
- Improves its insights from user feedback

The system aims to eliminate manual expense notes and provide a simple, intelligent money-tracking experience for students.

1.3 Definitions, Acronyms, and Abbreviations

- AI: Artificial Intelligence
- LLM: Large Language Model
- API: Application Programming Interface
- JWT: JSON Web Token

- RLS: Row Level Security
- SRS: Software Requirements Specification
- FR: Functional Requirement
- NFR: Non-Functional Requirement
- IR: Interface Requirement
- Budget Used: Percentage of a budget already spent, calculated from the recorded expenses

2. General Description

2.1 Product Perspective

Student Expense Tracker is a web-based application with the following architecture:

Frontend:

- React
- Vite
- Tailwind CSS

Backend:

- Supabase (Authentication, REST API, Row Level Security)

Tools:

- LLM (Gemini API) for spending insights
- The AI integration is isolated so the application still works if the AI API is unavailable.

Database:

- Supabase PostgreSQL

Deployment:

- Vercel

2.2 Product Functions

The system shall:

1. Allow users to register and log in.
2. Collect and manage a user profile and preferences.
3. Allow users to record, edit, and delete income and expense entries.
4. Track spending against monthly budgets (overall and per category).
5. Display daily, weekly, and monthly statistics with category-wise charts.
6. Generate simple AI spending insights.

2.3 User Classes and Characteristics

Primary User: Student

- Manages a limited allowance or income
- Basic to intermediate computer literacy
- Accessing the system through a web browser or mobile browser

Secondary User: Working Student

- Earns part-time income alongside studies
- Needs to track both income and expenses

Future User: Admin

- Manages default categories and platform configuration
- Monitors system health, analytics, and user activity

2.4 Operating Environment

- Web browser (Chrome, Edge, Firefox, Safari)
- Supabase PostgreSQL database
- LLM provider API (Gemini API)
- Hosting on Vercel

3. Functional Requirements

FR1: User Authentication

- The system shall allow user registration and login.
- JWT-based authentication shall be implemented through Supabase Authentication.
- OAuth 2.0 / Google sign-in shall be supported.
- Users shall be able to reset a forgotten password.

FR2: User Profile Management

- The system shall provide a simple onboarding interface to collect profile data.
- Profile fields shall include: name, preferred currency, monthly allowance or income, default monthly budget, and spending categories of interest.
- Users shall be able to update any profile field at any time.
- Users shall be able to delete their account and all associated data.

FR3: Expense and Income Recording

- The system shall allow users to add a transaction with amount, type (expense or income), category, date, payment method, and optional note.
- Users shall be able to edit and delete transactions.
- The system shall provide default categories and allow users to create custom categories.
- The system shall apply filters: date range, category, type, and payment method.
- The system shall validate that the amount is positive and the date is valid.
- Users shall be able to export their transactions as a CSV file.

FR4: Budget Tracking and Evaluation

- The system shall allow users to set an overall monthly budget and category-wise budgets.
- The system shall compute Budget Used (percentage) for every budget from the recorded expenses.
- The system shall show the remaining amount and the number of days left in the month.
- The system shall notify the user when spending reaches the alert thresholds (default 80% and 100%).
- The system shall allow users to customise alert thresholds via settings.

FR5: Dashboard and Report Output

- The system shall display today's, this week's, and this month's spending on the dashboard.
- The dashboard shall show the remaining monthly budget and the 5–10 most recent transactions.
- The system shall show category-wise spending charts and weekly and monthly spending trends.
- Each budget card shall display: Category, Budget Amount, Amount Spent, Remaining Amount, and Percentage Used.

FR6: AI-Based Spending Insights

The system shall generate simple insights including:

- Highest spending category for the selected period
- Comparison of this week's spending with last week's
- Suggestions for staying within a budget that is close to being exceeded

If the AI service is unavailable, all other features shall continue to work.

FR7: Feedback Learning

- When a user dismisses an insight, the system shall ask for a reason (not useful, incorrect, already known).
- The system shall use this feedback to avoid repeating similar insights.
- The system shall surface new insights without waiting for user action (weekly cadence).

4. Interface Requirements

IR1: User Interface

The interface shall include:

- User profile setup and onboarding panel
- Dashboard (/)
- Transactions page (/transactions)
- Budgets page (/budgets)
- Reports and AI insights page (/reports)
- Settings panel for profile, categories, and alert thresholds (/profile)

The UI must be clean, responsive, accessible (WCAG 2.1 Level AA), and user-friendly, with a sidebar on desktop and a mobile menu on small screens.

IR2: Software Interface

The system shall interact with:

- LLM (Gemini API)
- Supabase (database and authentication)

All communication uses REST and JSON.

IR3: Communication Interface

- All communication shall use HTTPS (TLS 1.3).
- Frontend and backend shall communicate via REST endpoints.
- The Gemini API key shall be stored in environment variables of the hosting platform and not in client-side code.

5. Design Constraints

The system must use:

- React with Vite (frontend)
- Tailwind CSS for styling

- Supabase for authentication, database, and REST API
- LLM (Gemini API) for AI processing
- Vercel-compatible deployment

6. Security Constraints

- TLS 1.3 encryption
- Encryption of stored data at rest (provided by the hosting platform)
- Follow GDPR and CCPA regulations

7. Non-Functional Attributes

NFR1: Security

- JWT authentication for all API requests
- OAuth 2.0 / OpenID Connect sign-in support
- Row Level Security (RLS) so users can access only their own records
- Input validation and sanitisation on all user inputs
- API keys and credentials stored in environment variables and secrets settings

NFR2: Scalability

- Stateless frontend deployed on Vercel
- Database indexes on user and date for fast transaction queries
- Modular AI service architecture — the AI provider can be changed without core code changes

NFR3: Reliability

- The application remains fully usable if the AI API is unavailable
- AI requests time out within 5 seconds and fall back to a friendly message
- Failed requests include retry logic with exponential back-off (max 3 retries)

NFR4: Maintainability

- Modular component-based architecture
- Clear separation of concerns (UI / services / database / AI)
- Automated tests for budget and statistics calculations
- Clean code practices and standardised project documentation

NFR5: Usability

- New user profile setup completable in under 2 minutes
- A first expense can be added in under 1 minute
- WCAG 2.1 Level AA accessibility compliance
- Supported languages: English

NFR6: Privacy

- User financial data is never sold to or shared with third-party advertisers
- Users can export all personal data or request permanent deletion within 30 days
- Deleted accounts fully purged from all systems within 90 days

NFR7: Performance Requirements

- Dashboard load time shall be less than 2 seconds.
- Adding a transaction shall update dashboard totals within 1 second.
- Budget and statistics computation shall not exceed 500 ms.
- AI insight generation shall complete within 8 seconds.
- CSV export of one year of transactions shall complete within 5 seconds.

8. Preliminary Schedule

8.1 Development Timeline

Requirement Analysis — 1 day

System Design & Database Schema — 2 days

User Profile & Authentication Module — 2 days

Transaction & Category Module — 3 days

Budget & Dashboard Module — 2 days

AI Insights & Notifications — 1 day

Frontend Development — 3 days

Integration and Testing — 2 days

Total Estimated Duration: 3 weeks

9. Appendices

A. System Architecture Overview

User → React Frontend → Supabase (Authentication + REST API) → PostgreSQL Database

React Frontend → Gemini API (spending insights)

- The frontend handles user interaction.
- Supabase handles authentication, data access, and security rules.
- The LLM (Gemini API) processes spending summaries into insights.
- The database stores system data.

B. Technology Stack

- Frontend: React, Vite, Tailwind CSS
- Backend: Supabase
- AI: Gemini API
- Database: Supabase PostgreSQL
- Deployment: Vercel

C. Supabase and Gemini API Reference

The system uses the Supabase auto-generated REST API to store and read data, and the Gemini API to generate spending insights.

API Overview

- Providers: Supabase, Google (Gemini)
- Cost: Free tier (subject to provider limits)
- API Key: Supabase public (anon) key in the frontend; Gemini key kept in environment variables
- Protocol: HTTPS / REST
- Response Format: JSON

Base URL

Supabase: <https://<project-ref>.supabase.co/rest/v1>

Gemini: <https://generativelanguage.googleapis.com/v1beta>

API Endpoints

1. Get All Transactions: GET /transactions
2. Filter by Date Range: GET /transactions?date=gte.<start>&date=lte.<end>
3. Filter by Category: GET /transactions?category_id=eq.<id>
4. Add Transaction: POST /transactions
5. Update Transaction: PATCH /transactions?id=eq.<id>
6. Delete Transaction: DELETE /transactions?id=eq.<id>
7. Get Budgets: GET /budgets?month=eq.<yyyy-mm>
8. Generate Insight: POST /models/<model>:generateContent (Gemini)

Response Fields

Each transaction object returned by the API contains the following fields:

- id – Unique transaction identifier
- user_id – Owner of the record
- category_id – Linked category
- type – expense or income
- amount – Transaction amount
- date – Date of the transaction
- payment_method – Cash / UPI / Card / Other
- note – Optional description
- created_at – Record creation time

API Documentation

- Supabase: <https://supabase.com/docs>
- Gemini API: <https://ai.google.dev>

All links use HTTPS protocol.

D. Assumptions

- Users have internet connection.
- Supabase and Vercel services are available and accessible.
- LLM service is active.
- Users provide correct information.

E. Future Enhancements

- Mobile application version.
- Recurring expenses and reminders.
- Shared expenses and bill splitting with friends.
- Receipt scanning.
- Advanced analytics dashboard.

10. Conclusion

The Student Expense Tracker is an intelligent money-tracking system designed to simplify and improve how students manage their spending. It lets students record income and expenses, tracks them against monthly budgets, and presents clear daily, weekly, and monthly statistics with category-wise charts. The system then generates simple AI insights with clear reasons, helping students make better spending decisions. By reducing manual calculation and making spending patterns visible, the Student Expense Tracker saves time, minimizes effort, and provides a personalised and efficient expense-tracking experience.